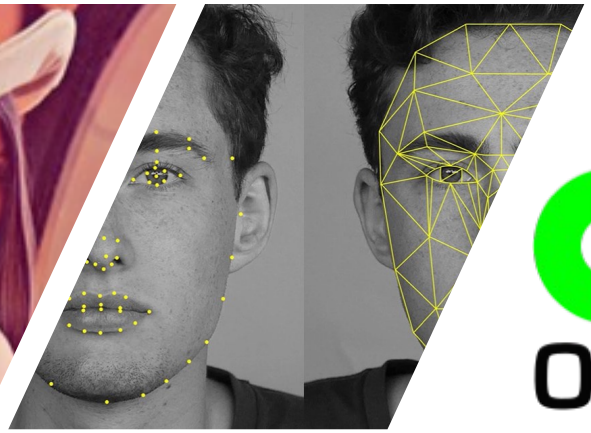
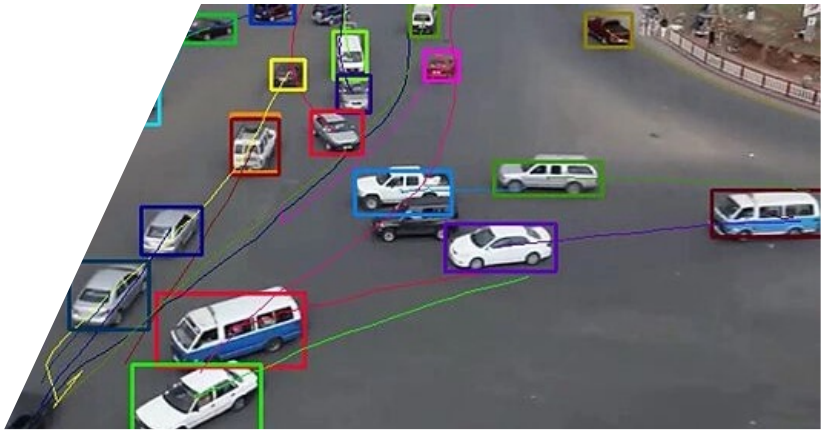
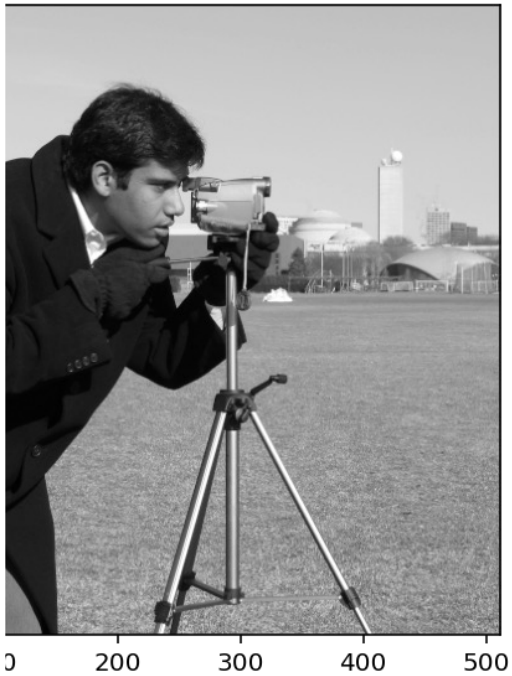
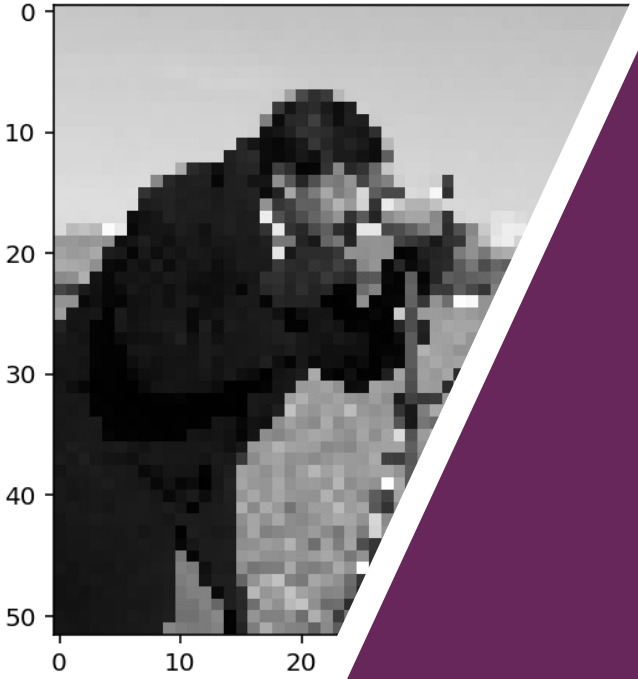


OpenCV







ECE 1390/2390

Image Processing and Computer Vision

About this course

Course number 1390 (29 students)

Course number 2390 [Graduate level] (12 students)

- graduate level will have harder problem sets and more involved project goals

<u>Learning Objectives</u>:

- Introduction to image processing methods including data structures, filtering, enhancement, transforms, and homography.
- Introduction to computer vision through OpenCV and Python including object detection and tracking methods, segmentation, digital photography, and in-painting.
- Hands on learning through group project

Who am I?

Instructor: Ted Huppert, PhD;
Associate Professor (ECE)

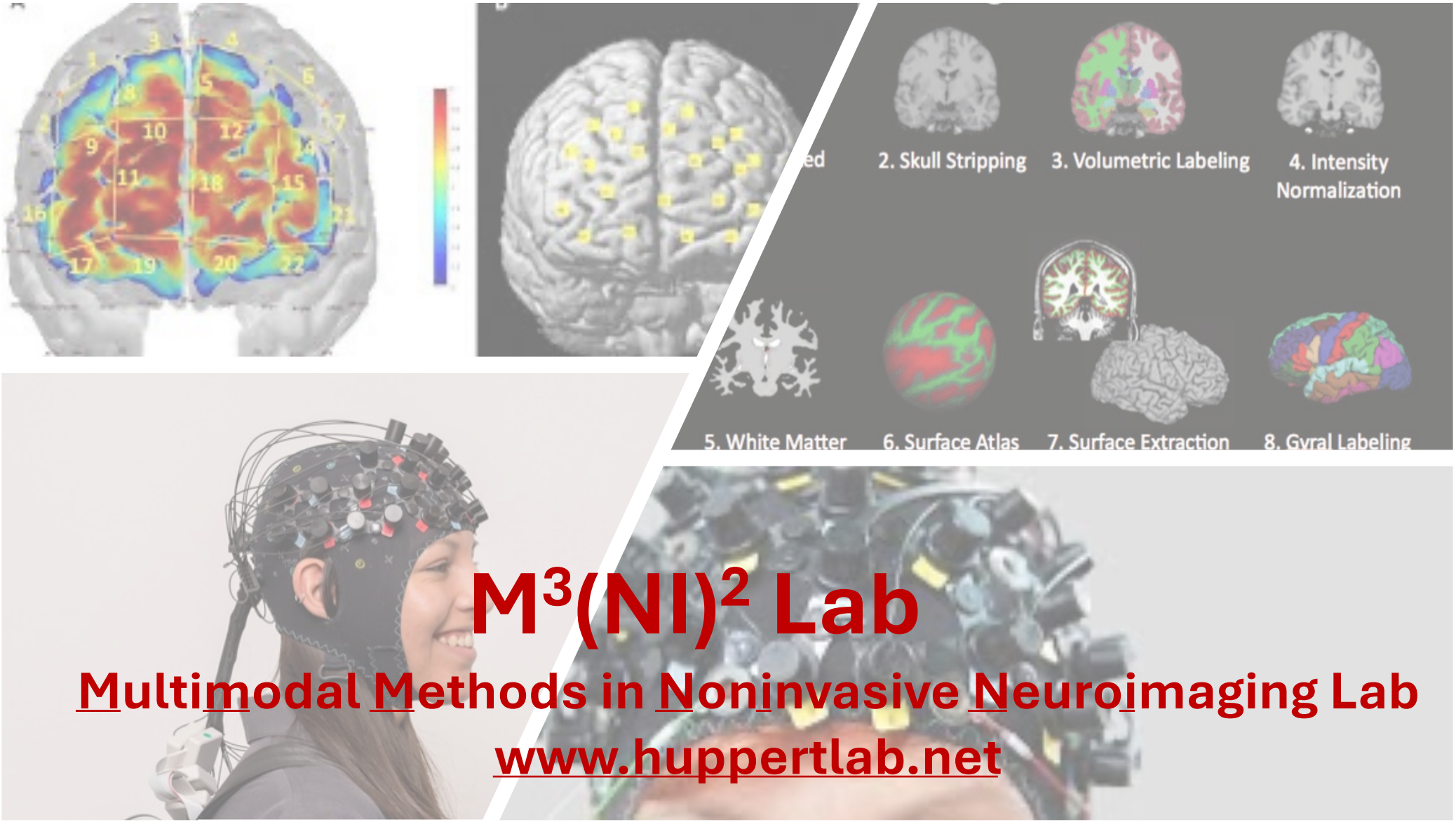
Dept of Radiology (2007-2019)
Dept of ECE (2019-current)

Email: huppert1@pitt.edu
Office: BEND 1238L (in 12th floor ECE admin area)
Office hours: M/W 3:30-4:30
https://calendly.com/huppert1

Who am I?

Teaching Instructor:

Difei Tang



$M^3(NI)^2$ Lab

Multimodal Methods in Noninvasive Neuroimaging Lab

www.huppertlab.net

Organization of Course

- Monday & Wednesday 4:30 – 5:45pm
- 1211 BEND

- **Lectures** (PowerPoint & Python Notebooks)
  - Materials will be provided on Canvas
  - No required textbook for course
- **Classwork** (40% of grade)
  - Practical examples done in Python/OpenCV
  - Done mostly in class in small groups. If not finished, then homework
- **Semester project** (20% final; 20% peer-ratings; 20% in class)
- **No midterm or Final exams**

Grading Scale:

Homework	40% [12 x 10 points each]
Semester Project Peer scores	20% [60 pts]
In class group participation	20% [60 pts]
Final Semester Project	20% [60 pts]

	Total 240 points

[98 - 100) A+	[88 - 90) B+	[78 - 80) C+	[68 - 70) D+
[92 - 98) A	[82 - 88) B	[72 - 78) C	[62 - 68) D
[90 - 92) A-	[80 - 82) B-	[70 - 72) C-	[60 - 62) D-
<60: F			

Syllabus

* Lecture will be remote
** Lecture will be pre-recorded

Week	Date	Topic	In class work	Due date
1	8/26/2024	Introduction to course	Intro to OpenCV	
	8/28/2024	Image operations	Planning projects	
2	9/2/2024	Labor Day [No Class]		
	9/4/2024	Analysis and Color spaces	Initial project proposal	
3	9/9/2024. *	Image Transforms	Python problems (HW1)	Due 9/16
	9/11/2024 *	Spatial & Frequency Filtering	Group Project work	White paper on project
4	9/16/2024 **	Image Restoration/Inverse filters	Python problems (HW2)	Due 9/23
	9/18/2024	Edge Detection	Group Project work	
5	9/23/2024	GMM Segmentation	Python problems (HW3)	Due 9/30
	9/25/2024	Morphometric Segmentation	Python problems (HW4)	Due 10/2
6	9/30/2024	Image Feature Detection	Group Project work	
	10/2/2024	Registration & Homography	Python problems (HW5)	Due 10/9
7	10/7/2024	Computational Photography	Python problems (HW6)	Due 10/16
	10/9/2024	Inpainting	Mid project assessment	Group ratings
8	10/14/2024	FALL Break [No Class]		

Syllabus

Week	Date	Topic	In class work	Due date
8	10/14/2024	FALL Break [No Class]		
	10/16/2024	Camera Calibration	Python problems (HW7)	Due 10/23
9	10/21/2024	Image Compression	Group Project work	
	10/23/2024	Depth & 3D Reconstruction	Python problems (HW8)	Due 10/30
10	10/28/2024	Haar Cascade Classifiers	Python problems (HW9)	Due 11/4
	10/30/2024	Face Detection	Project updates	
11	11/4/2024	Harris Corners and Blobs	Group Project work	
	11/6/2024	HOG and Custom Detectors	Python problems (HW10)	Due 11/13
12	11/11/2024	Object Tracking	Group Project work	
	11/13/2024	OCR Text Detection	Python problems (HW11)	Due 11/20
13	11/18/2024	OpenCV DNN	Python problems (HW12)	Due 12/4
	11/20/2024	Super Resolution	Python problems	
14	11/25/2024	Thanksgiving recess		
	11/27/2024	Thanksgiving recess		
15	12/2/2024	SLAM	Group Project work	
	12/4/2024	MediaPipe	Group Project work	Final project commit
16	12/9/2024	Final Projects	Project presentations	Final group ratings

Semester Project

Design a program to implement image processing.

- Python function library of methods
- Can be GUI or command line
- Groups of 4 people (1390/2390 separated)

Examples:

- Virtual web camera
- Face-swap
- Object detection
- Text reader
- “Instagram” filter
- 3D reconstruction
- Automatic face blurring software

Requirements:

Everyone

- Written in Python using OpenCV
- Code compliant with MIT use license
- Documented and maintained on GitHub
- Include demonstration/example code

ECE 1390 Students

- Load/Save and process still images

- include an image enhancement method
- Include an image filtering method
- Include an edge detection method
- Demonstrate segmentation
- Include some object recognition

- Incorporate 1 additional methods from class code

ECE 2390 Students

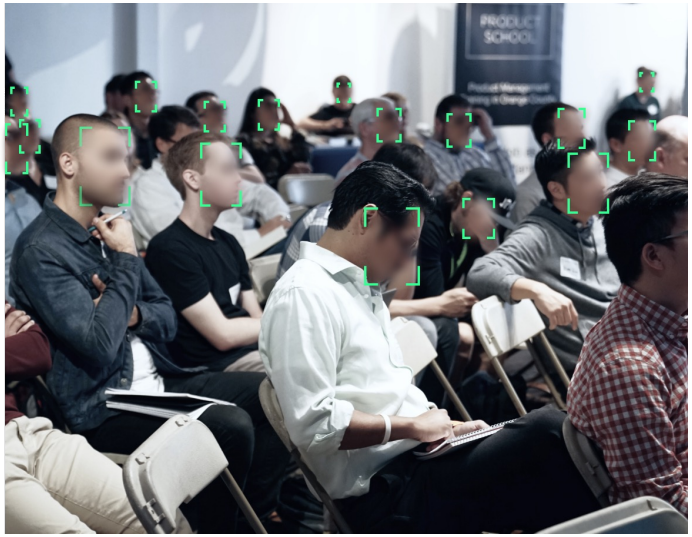
- Load/Save and process still images
- Process video feed
- include an image enhancement method
- Include an image filtering method
- Include an edge detection method
- Demonstrate segmentation
- Include some object recognition
- Include some object tracking
- Incorporate 3 additional methods from class code

Example. Automatic facial detection and blurring

<u>ECE 1390 Students</u>

- Load/Save and process still images → Load portrait image
  → Save cropped image
- include an image enhancement method → Auto adjust saturation
- Include an image filtering method → Apply smoothing
- Include an edge detection method → Show edge contours

- Include some object recognition → Detect face
- Demonstrate segmentation → Remove background
- Incorporate 1 additional methods from class code → Sharpen
  → Change color scheme

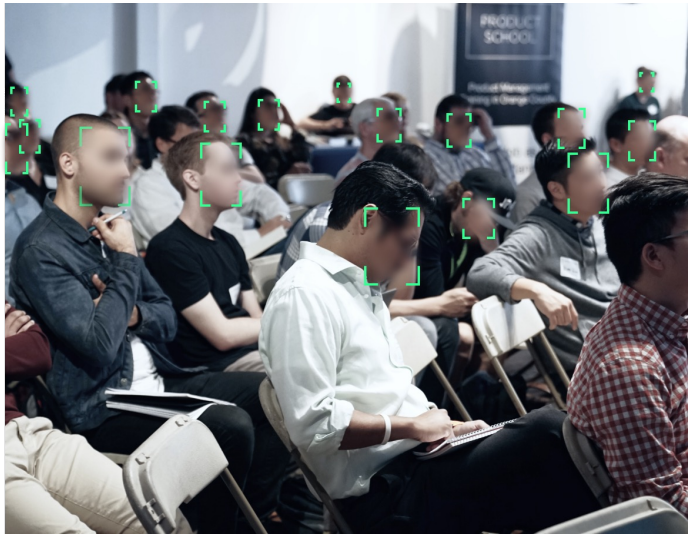


https://www.plugger.ai/blog/automated-human-face-blurring-for-privacy

Example. Automatic facial detection and blurring

ECE 2390 Students

- Load/Save and process still images → Get from webcam
- Process video feed → Send to Zoom
  → Still frame capture
- include an image enhancement method → Auto adjust saturation
- Include an image filtering method → Apply smoothing
- Include an edge detection method → Show edge contours
- Demonstrate segmentation → Remove background
- Include some object recognition → Detect face
- Include some object tracking → Track face over video
- Incorporate 3 additional methods from class code → Sharpen
  → Change color scheme
  → Apply camera calibration
  → Increase resolution



https://www.plugger.ai/blog/automated-human-face-blurring-for-privacy

Group Assessments

20% Project peer score

- Part of your grade will be based on how your other teammates feel you contributed. At the mid-point and end of the course there will be a rubric to score your project peers.

20% Class peer score

- Part of your grade will be based on your peers' feedback on your project.

20% Final project

- Did you meet the requirements?
- Is the code documented and annotated?
- Was GitHub used effectively?

Project “Investor” Pitch (Due 9/4/2024)

- Each group will have 2 min to pitch their idea. (“elevator pitch”)
- 1 PowerPoint slide may be used (sent to Dr Huppert prior to class)
- Class will have 5min to ask questions, make suggestions, get clarifications, etc. (think focus group feedback)

- At the end, there will be a survey (via canvas) where you award “money” to the group that you would invest in.

Project White page (Due 9/11/2024)

- See full description on canvas

One-page white paper describing purpose and specs of your project.

- Description.
- Code Specifications.
- Planned approach
- Time-line
- Metrics of success.
- Pitfalls and alternative solutions

Yes, you can borrow code and methods.

- With OpenCV and Python, you can find examples of virtually any code/project on the web.
- However, I grading on whether you demonstrate that you understood it.

- Requirements:
  - If you borrow code or use some tutorial to learn how to do the problem, then CITE IT!
  - Make the code your own. If you borrow code, then change variable names to be consistent with the rest of your code/library
  - Markup your notebooks and code with proper web-links
  - Implement best practices in coding
    - Names for variables/methods should be descriptive
    - Code should succinctly and clearly explain what it does.
    - Methods should document expected inputs/outputs/dependencies
    - Yes, it's fine to comment (e.g.) "I don't know why this flag is TRUE, but it doesn't work otherwise"
    - Make sure someone else could read and understand your code
  - Use GitHub. Document changes, do pull requests, issue reports when needed.

Yes, you can borrow code and methods.

You must borrow code for your project!

Semester Project requires that you "incorporate 1or 3 additional methods from class code". Oct 30th class.

We will all be using GitHub classroom for our semester projects. I want you to use methods from someone else's library in your project.

- Do a GitHub pull request
- Cite the other code
- If the documentation is unclear or the code doesn't do what it says it should, then submit a GitHub issue report.
- If someone posts an issue on your code, fix it! Then request a pull review from the person who issued the problem report.
- If the code does what it is supposed to, but you need to modify it anyway to work with your program, then submit your edits with a push request.

Frequently asked questions

- Can I do a video project even though I am enrolled in 1360?
  - Yes. You have to do the project requirements for 1360, but if you want to add the additional ability to do video or object tracking, you absolutely can. You will get up to 20pts extra credit if you do the extra work.
- I have a great idea of methods to add, but it wasn't mentioned. Can I implement additional methods to my project?
  - Yes. Same as above. You need to do the project requirements, but if you want to add something extra, you can get up to 20pts extra credit. You need to get permission from the instructor, and it needs to be a challenging enough addition. This applies to 1360 and 2360 students.

Frequently asked questions

- What is your late work policy?
  - You can turn in homework late for half credit. To stay on schedule with the course and project, it is important to keep up with the schedule, but I know things come up. You need to turn in the work by the end of the semester.

- What if I don't finish the semester project?
  - If you keep up with the milestones, you shouldn't have any problem finishing the minimum requirements, but you might not finish everything you planned to and wrote about in your white paper proposal of the project. In that case, I expect to see this documented on the white paper (which is part of the GitHub project).
  - I would rather you try too much and have to scale back, then to do the bare minimum.

Frequently asked questions

- Can I work on a language other than Python?
  - No. I really expect this to be done in Python. Matlab's version of OpenCV is very limited. If you have trouble with Python, there are many online resources, and we can help you in office hours.

- What if I don't know how to use GitHub?
  - Part of developing software for a company is making sure that the code is documented and tracked for the next person to use. These are skills that I want you to learn (or improve) in this course.

Side note on licenses*

* It is a requirement of your projects that you pay attention to these

Permissive License

- Apache license (OpenCV)
- MIT license
- BSD license

- Can distribute, modify, and distribute modified versions without royalty
- Allows proprietization (you can use it as part of a more restrictive license)
- Requires NOTICE file to be included in all derivatives
- MIT requires statement of “As Is” and original copyright notice to be included in the documentation.

Copyleft

- GPL license
- AGPL license

- Can distribute, modify, and distribute modified versions without royalty
- No proprietization (derivatives must keep the same license as the original)
- Requires NOTICE file to be included in all derivatives

Non-commercial

- JRL license
- AFPL license

- Can only be used for non-commercial use

Public Domain

- PD license
- CC0 license

- No restrictions at all

Proprietary

- Can’t use!

Questions?

Types of Images

- Binary
  - 0 & 1
  - Used for masks
  - 1-bit per pixel

- Black & White image
  - 0 & 1
  - At least 8bit per pixel
  - Display provides smoothness

Although Binary and B&W both encode the same information (only 1 & 0's), the bit-depth changes how they are handled by display functions.

Types of Images

- Greyscale
  - Pixel values across range
  - At least 8bit per pixel
  - One color channel

- Color
  - Pixel values across range
  - At least 8bit per pixel
  - multiple color channels (typically 3 or 4)

Typically the
“alpha” channel

Glossary:

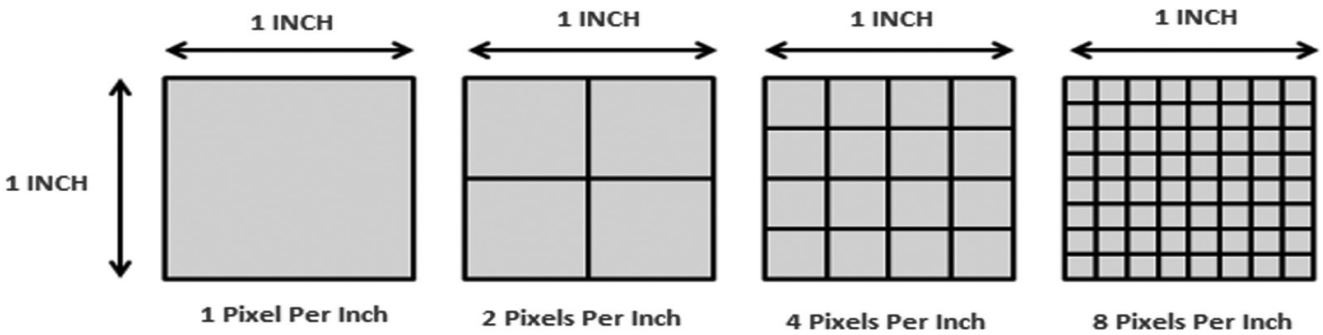
BPP: Bits per Pixel (bit depth)

Bits per Pixel (bpp)	Number of Colors
1 bpp	2 colors
2 bpp	4 colors
3 bpp	8 colors
4 bpp	16 colors
5 bpp	32 colors
6 bpp	64 colors
7 bpp	128 colors
8 bpp	256 colors
10 bpp	1024 colors
16 bpp	65,536 colors
24 bpp	16,777,216 colors (16.7 million colors)
32 bpp	4,294,967,296 colors (4294 million colors)

Bit value is an index! You always also need to define a color map (e.g. RGB, HSV, etc)

Glossary:

- <u>Pixel Density</u>



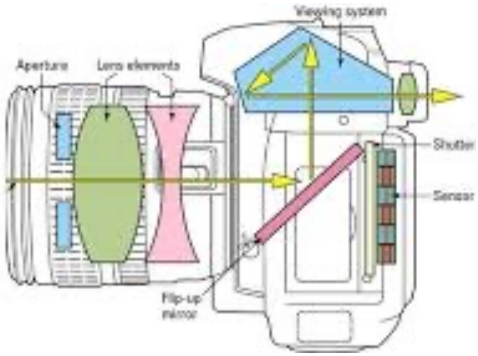
Humans can't differentiate details beyond around 300 PPI

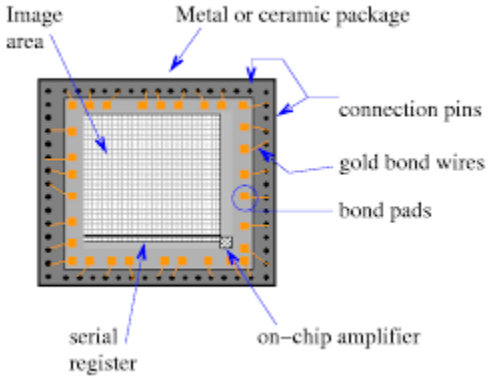
- <u>Resolution</u>

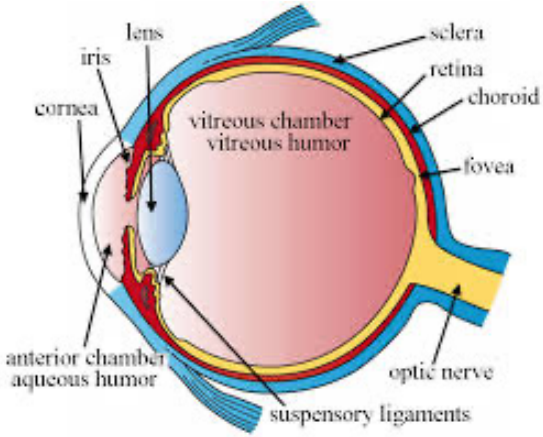
  Bit more ambiguous of a term.
  Sometimes:
  - total number of pixels (e.g. 3Mpixels)
  - dimensions (e.g. 2556 x 1179)
  - PPI (e.g. 460ppi)
  - Pixels/degree (e.g. Apple Retina™ display)

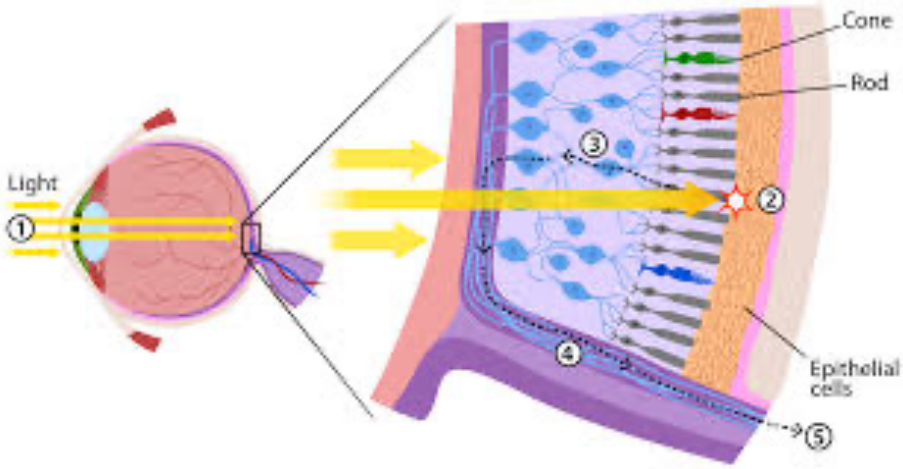
Glossary:

Resolution



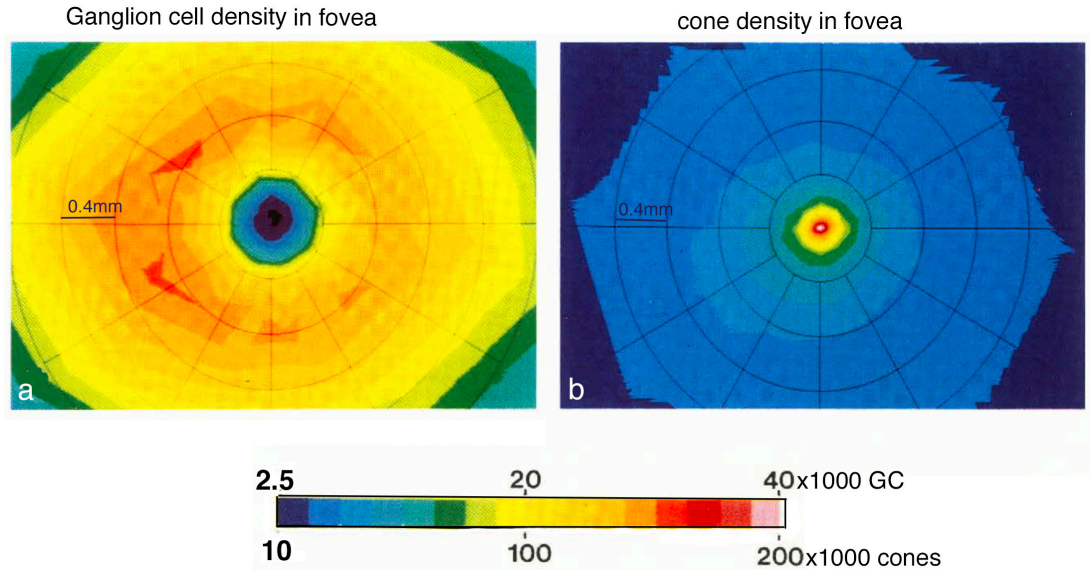






Glossary:

Resolution



From Curcio and Allen 1990

- Visual acuity is highest at the center of the eye (fovea)
- Here, the limit of the human eye is about 60 pixels / degree (1 arc-minute)
  e.g. 3600 pixels (60 x 60) in a 1° x 1° area
- Depends on distance from object

Glossary:

Resolution

60 px/degree

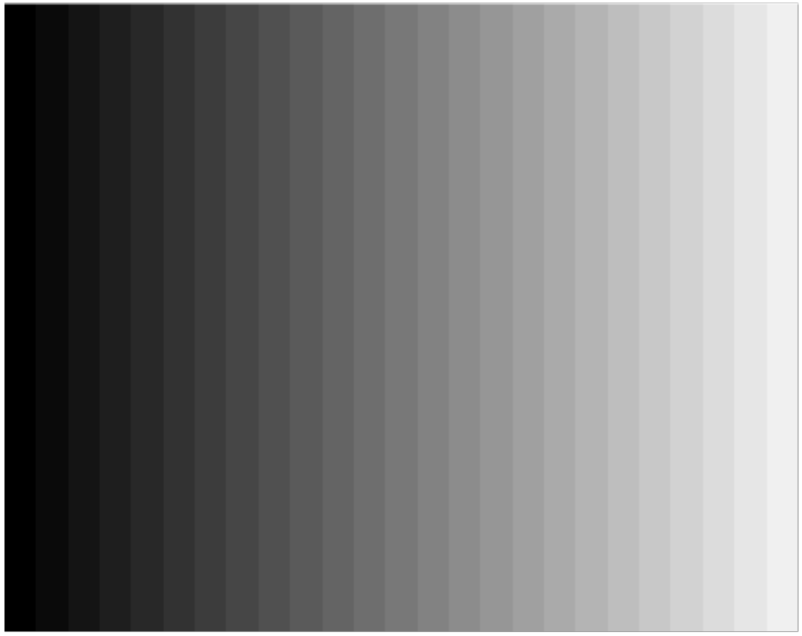
30 px/degree

15 px/degree

7.5 px/degree

Glossary:

- Intensity
  - How large is the pixel value?
- Brightness
  - How does the eye perceive intensity
- Contrast
  - Range of intensities of pixels



Harder to differentiate

Easier to differentiate

Harder to differentiate

Gamma correction (and similar) designed to adjust intensity → brightness (next lecture)

Glossary:

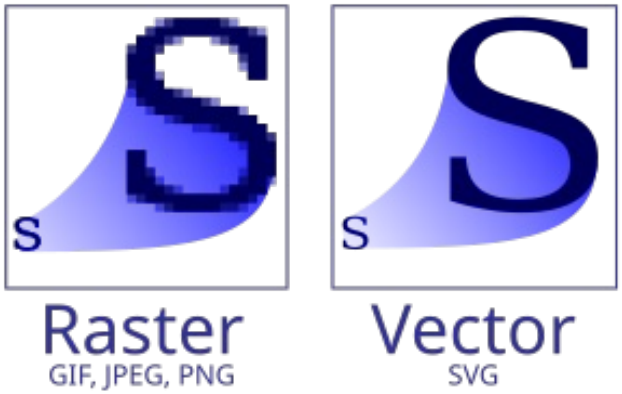
<u>Raster images</u>

-Use pixels.

-Lose information when resizing/zooming

-E.g.

JPEG, PNG, TIFF

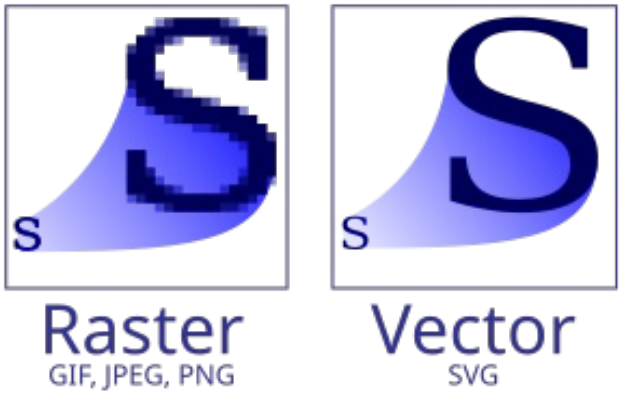


Glossary:

<u>Vector images</u>

-Define the image using piecewise formulas.
-Lossless when resizing/zooming
-Converted to raster for display

-E.g.
SVG, EPS, PDF, AI, PSD



Glossary:

Tiff (Tag Image File Format)

- Raster image format
- Most versatile for colors
- Supports RGB, CMY, Grayscale, and more
- Uncompressed (large file sizes, but lossless)

JPEG (Joint Photographic Experts Group)

- Raster image format
- Compressed image data (smaller size, but degradation)

GIF (Graphics Interchange Format)

- Raster image format
- Compressed color maps (smaller size, less color contrast)

OpenCV (https://opencv.org/)

- Developed by Intel in 1999
- Open-source (https://github.com/opencv)
- Apache License 2

Features:

- Image processing and visualization
- Object recognition
- Segmentation
- Facial and gesture recognition
- Image homography/registration
- Depth perception
- Deep learning methods

Code:

- Written in C++
- Bindings in Python, Java, Matlab
- CUDA support (since 2010)
- Currently version 4